Prathamesh Deshmukh Assignnment 6

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML_Data/Heart_Failure_Prediction_dataset - Heart_Failure_Prediction_dataset.csv"
df=pd.read_csv(path)
df.head(5)
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
df.shape
```

```
(918, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
df.describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

```
df.columns
```

```
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
       'HeartDisease'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.isnull().sum()
```

	0
Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

dtype: int64

```
df[df.isnull().any(axis=1)]
```

Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease

```
X = df.iloc[:,3:-1]
y = df['HeartDisease']
```

```
X.head()
```

	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
0	140	289	0	Normal	172	N	0.0	Up
1	160	180	0	Normal	156	N	1.0	Flat
2	130	283	0	ST	98	N	0.0	Up
3	138	214	0	Normal	108	Y	1.5	Flat
4	150	195	0	Normal	122	N	0.0	Up

```
X = pd.get_dummies(X, dtype=int)
```

```
X.shape
```

```
(918, 13)
```

```
X.head(5)
```

	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	Exercis
0	140	289	0	172	0.0	0	1	0	1	
1	160	180	0	156	1.0	0	1	0	1	
2	130	283	0	98	0.0	0	0	1	1	
3	138	214	0	108	1.5	0	1	0	0	
4	150	195	0	122	0.0	0	1	0	1	

```
X.columns
```

```
Index(['RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',
       'RestingECG_LVH', 'RestingECG_Normal', 'RestingECG_ST',
       'ExerciseAngina_N', 'ExerciseAngina_Y', 'ST_Slope_Down',
       'ST_Slope_Flat', 'ST_Slope_Up'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train,y_test = train_test_split(X,y,train_size=0.7,random_state=42)
```

```
X_train.shape,X_test.shape
```

```
((642, 13), (276, 13))
```

```
X_train.head()
```

	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	Exerc
712	100	234	0	156	0.1	0	1	0	1	
477	110	0	1	108	2.0	0	1	0	0	
409	160	0	1	122	0.0	1	0	0	0	
448	160	230	1	105	1.0	0	1	0	0	
838	130	330	1	132	1.8	1	0	0	0	

```
from sklearn.preprocessing import StandardScaler
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

```
array([[-1.77231378,  0.2949892 , -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [-1.23486848, -1.90539005,  1.90228942, ...,  3.93211628,
        -1.02524056, -0.86288193],
       [ 1.45235803, -1.90539005,  1.90228942, ..., -0.25431598,
         0.97538084, -0.86288193],
       ...,
       [-0.15997788,  0.47365248, -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [ 1.02240179, -1.90539005, -0.52568236, ..., -0.25431598,
        -1.02524056,  1.15890711],
       [ 0.91491273,  1.78071537, -0.52568236, ..., -0.25431598,
         0.97538084, -0.86288193]])
```

```
import tensorflow as tf
```

```
ann = tf.keras.models.Sequential()
```

```
ann.add(tf.keras.layers.Dense(units=9,activation='relu'))
```

```
ann.add(tf.keras.layers.Dense(units=1,activation='sigmoid'))
```

```
ann.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
```

```
ann.fit(X_train,y_train,batch_size=32,epochs=100)
```

Show hidden output

```
y_pred=ann.predict(X_test)
y_pred=(y_pred > 0.5)
y_pred
```

Show hidden output

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
cm=confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
[[ 90  21]
 [ 22 143]]
```

```
print('Accuracy Score')
print(accuracy_score(y_pred,y_test))
```

```
Accuracy Score
0.8442028985507246
```

```
print('Classification Report')
print(classification_report(y_pred,y_test))
```

```
Classification Report
              precision    recall  f1-score   support

       False       0.80      0.81      0.81       111
        True       0.87      0.87      0.87       165

    accuracy                           0.84       276
   macro avg       0.84      0.84      0.84       276
weighted avg       0.84      0.84      0.84       276
```